



Números (pseudo)aleatorios y Generación de Variables Aleatorias

Miguel Ángel Rincón Morales¹

1. Cod: 160005038, Ing. Sistemas.

Facultad de Ciencias Básicas e Ingenierías. Programa de Ingeniería de Sistemas.

Fecha: 4 de Septiembre de 2026

Resumen

En este laboratorio programé en Python, sobre Google Colab, cuatro generadores de números pseudoaleatorios — MidSquare, congruencial mixto, Lagged-Fibonacci y Shift-Register — y a cada secuencia le medí el tamaño de ciclo, la uniformidad con el contraste χ^2 y la aleatoriedad con el test de rachas. Sólo los congruenciales de módulo grande combinaron un periodo largo con la aprobación de las dos pruebas; MidSquare, en cambio, degeneró con tres de las cinco semillas ensayadas. Con el mejor de los generadores aproximé por Monte Carlo las siete integrales de los ejercicios 3 a 9 del Capítulo 3 de [Ross99] y generé variables aleatorias discretas por transformada inversa y por composición. Por último repetí la simulación *ad hoc* del Laboratorio 1 usando llegadas Poisson y servicios Binomial, con un resultado bastante menos congestionado que el original.

Palabras clave — números pseudoaleatorios, generador congruencial, tamaño de ciclo, prueba χ^2 , test de rachas, método Monte Carlo, transformada inversa, método de composición.

1. Objetivos

- Introducir los métodos para la generación y validación de números pseudoaleatorios.
- Generar secuencias de números pseudoaleatorios usando el método de MidSquare.
- Generar secuencias de números pseudoaleatorios usando el método del congruencial mixto.
- Generar secuencias de números pseudoaleatorios usando otros métodos (Lagged-Fibonacci y Shift-Register).
- Calcular el tamaño de ciclo o periodo de una secuencia de números pseudoaleatorios.
- Evaluar si una secuencia de números pseudoaleatorios cumple con los criterios de uniformidad y aleatoriedad.
- Aproximar integrales definidas, impropias y múltiples mediante el método Monte Carlo.
- Generar secuencias de valores de variables aleatorias discretas usando el método de la transformada inversa y el método de composición.



2. Fundamento teórico

2.1. Generadores de números pseudoaleatorios

Un generador de números pseudoaleatorios es un algoritmo determinista $s_{i+1} = T(s_i)$ sobre un espacio de estados finito, del que se extrae una salida $u_i \in (0, 1)$. Como los estados son finitos, toda secuencia termina repitiéndose: existen una *cola* μ y un *ciclo* λ tales que $s_{i+\lambda} = s_i$ para todo $i \geq \mu$. Lo que se busca es que λ sea muy grande y que los u_i pasen por una muestra i.i.d. $U(0, 1)$ [1].

MidSquare (cuadrados medios). Propuesto por von Neumann. A partir de una semilla X_0 de d dígitos:

$$X_{i+1} = \text{dígitos centrales}(X_i^2), \quad u_{i+1} = \frac{X_{i+1}}{10^d} \quad (1)$$

Generador congruencial mixto o lineal (GLC). Es el método estándar en la práctica [2, 3]:

$$X_{i+1} = (a X_i + c) \text{ mód } m, \quad u_{i+1} = \frac{X_{i+1}}{m} \quad (2)$$

donde a es el multiplicador, c el incremento y m el módulo. El teorema de Hull-Dobell establece que el GLC alcanza el periodo máximo m si y sólo si: (i) $\gcd(c, m) = 1$; (ii) $a - 1$ es divisible por todos los factores primos de m ; y (iii) si $4 \mid m$, entonces $4 \mid (a - 1)$. Cuando $c = 0$ el generador se llama *multiplicativo* y su periodo máximo baja a $m - 1$, alcanzable sólo si m es primo y a es raíz primitiva módulo m .

Lagged-Fibonacci. Generaliza la sucesión de Fibonacci usando dos retardos $j < k$:

$$X_i = (X_{i-j} * X_{i-k}) \text{ mód } m, \quad * \in \{+, -, \times, \oplus\} \quad (3)$$

Requiere k semillas iniciales y su estado es la ventana de los últimos k valores, por lo que su periodo puede ser muchísimo mayor que m .

Shift-Register (LFSR). Opera sobre una cadena de bits mediante una recurrencia con la operación *o exclusivo*, $b_i = b_{i-p} \oplus b_{i-q}$; cada bloque de n_b bits consecutivos forma un entero. Si el polinomio característico $x^p + x^q + 1$ es primitivo sobre $GF(2)$, el periodo de la cadena de bits es $2^p - 1$.

2.2. Tamaño de ciclo y pruebas estadísticas

(a) **Tamaño de ciclo.** Se determina guardando los estados visitados hasta dar con el primero que se repite. La detección tiene que hacerse sobre el estado completo del generador y no sobre la salida X_i : en Lagged-Fibonacci y Shift-Register el estado es una ventana de varios valores, así que un X_i repetido no significa que la secuencia vuelva a empezar.

(b) **Prueba de uniformidad: contraste χ^2 .** Se parte el intervalo $(0, 1)$ en k clases disjuntas de igual amplitud $1/k$. Con O_i el número de observaciones en la clase i y $E_i = n/k$ el número esperado bajo H_0 :

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i} \sim \chi_{k-1}^2 \quad (4)$$

Se rechaza la hipótesis de uniformidad si $\chi^2 > \chi_{k-1, \alpha}^2$. En este laboratorio $k = 10$, $n = 100$ (por lo que $E_i = 10 > 5$, condición de validez de la aproximación) y $\alpha = 0,05$, lo que da un valor crítico $\chi_{9; 0,05}^2 = 16,92$.



(c) **Prueba de aleatoriedad: test de rachas** [4]. A partir de la muestra $u_1, \dots, u_n$ se construye una sucesión de signos comparando cada valor con el anterior (0 si $u_{i+1} < u_i$, 1 en caso contrario). Cada grupo maximal de símbolos iguales es una *racha*. Para n suficientemente grande, el número de rachas R se distribuye aproximadamente como

$$R \sim \mathcal{N}\left(\frac{2n-1}{3}, \sigma^2 = \frac{16n-29}{90}\right), \quad Z = \frac{R - \frac{2n-1}{3}}{\sqrt{\frac{16n-29}{90}}} \quad (5)$$

Se rechaza la hipótesis de aleatoriedad si $|Z| > Z_{\alpha/2}$; para $\alpha = 0,05$, $Z_{0,025} = 1,96$. Un Z muy negativo significa pocas rachas, o sea tendencias y bloques monótonos; uno muy positivo, demasiadas rachas, que es oscilación artificial.

2.3. Uso de números aleatorios para evaluar integrales

Sea $\theta = \int_0^1 g(x) dx$. Si $U \sim U(0, 1)$ entonces $\theta = E[g(U)]$, y si $u_1, \dots, u_n$ son i.i.d. $U(0, 1)$, la ley fuerte de los grandes números garantiza [2]

$$\hat{\theta}_n = \frac{1}{n} \sum_{i=1}^n g(u_i) \xrightarrow[n \rightarrow \infty]{} \theta \quad \text{con probabilidad 1} \quad (6)$$

Este es el método Monte Carlo. Su error estándar es $\sigma_g/\sqrt{n}$: la precisión mejora con $\sqrt{n}$ y no depende de la dimensión del integrando, lo que lo vuelve muy útil en integrales múltiples. Los cambios de variable que llevan cualquier dominio a $(0, 1)$ son:

$$\int_a^b g(x) dx = \int_0^1 (b-a) g(a + (b-a)y) dy \quad (7)$$

$$\int_0^\infty g(x) dx = \int_0^1 \frac{g\left(\frac{1}{y} - 1\right)}{y^2} dy \quad \text{con } y = \frac{1}{x+1} \quad (8)$$

2.4. Generación de variables aleatorias discretas

Método de la transformada inversa [2, 5]. Para una variable discreta con masa $p_j = P\{X = x_j\}$ se genera $U \sim U(0, 1)$ y se asigna

$$X = x_j \quad \text{si} \quad \sum_{i < j} p_i \leq U < \sum_{i \leq j} p_i \quad (9)$$

lo cual es correcto porque $P\{a \leq U < b\} = b - a$. Como el algoritmo tarda en proporción al número de intervalos que examina, conviene recorrer los valores en orden decreciente de p_j .

Generación de una v.a. Poisson (sección 4.2 de [2]). X es Poisson de parámetro $\lambda > 0$ si

$$P\{X = i\} = \frac{\lambda^i e^{-\lambda}}{i!}, \quad i = 0, 1, 2, \dots \quad (10)$$

con $E[X] = \text{Var}(X) = \lambda$. Aprovechando la recursión $p_{i+1} = \frac{\lambda}{i+1} p_i$:

PASO 1: Generar U . PASO 2: $i = 0$, $p = e^{-\lambda}$, $F = p$. PASO 3: Si $U < F$, hacer $X = i$ y terminar. PASO 4: $p = \lambda p / (i + 1)$, $F = F + p$, $i = i + 1$. PASO 5: Ir al paso 3.



Generación de una v.a. Binomial (sección 4.3 de [2]). X es binomial de parámetros n y p si cuenta el número de éxitos en n ensayos de Bernoulli independientes:

$$P\{X = i\} = \binom{n}{i} p^i (1-p)^{n-i}, \quad i = 0, 1, \dots, n \quad (11)$$

con $E[X] = np$ y $\text{Var}(X) = np(1-p)$. Con la recursión $p_{i+1} = \frac{n-i}{i+1} \frac{p}{1-p} p_i$:

PASO 1: Generar U . PASO 2: $c = p/(1-p)$, $i = 0$, $pr = (1-p)^n$, $F = pr$. PASO 3: Si $U < F$, hacer $X = i$ y terminar. PASO 4: $pr = [c(n-i)/(i+1)] pr$, $F = F + pr$, $i = i + 1$. PASO 5: Ir al paso 3.

Método de composición (sección 4.5 de [2]). Si la masa objetivo puede escribirse como mezcla de dos masas más fáciles de simular,

$$p_j = \alpha q_j + (1-\alpha) r_j, \quad 0 < \alpha < 1 \quad (12)$$

entonces basta generar U_1 y simular a partir de q si $U_1 < \alpha$ o a partir de r en caso contrario.

3. Procedimiento o metodología

El desarrollo completo está en el Notebook `Lab2_160005038.ipynb`, ejecutado en Google Colab y adjunto a este informe. No usé ninguna función de aleatoriedad de librería: los números aleatorios del trabajo salen de los generadores que programé. De `scipy` tomé únicamente `stats`, para los valores críticos y los p -valores, y `integrate` como referencia de comparación en la sección de integrales.

3.1. Generación de las secuencias (5.1.1, 5.1.2 y 5.1.3)

Programé cuatro generadores y produje con ellos doce secuencias de $n = 100$ números: cinco con MidSquare (semillas 1931, 675248, 4567, 12122 y 7179345), cinco con el generador congruencial mixto usando los cinco juegos de parámetros de la guía, y dos con los métodos adicionales que tomé de [3]: Lagged-Fibonacci con $j = 5$, $k = 17$, $m = 2^{31}$ y 17 semillas, y Shift-Register con $p = 7$, $q = 3$ y bloques de $n_b = 10$ bits. Cada secuencia quedó en una tabla de `pandas` con sus u_i .

3.2. Reformulación de los generadores para el análisis (5.1.4)

Al principio detectaba el ciclo comparando únicamente la salida X_i , que es lo primero que se le ocurre a uno. Con MidSquare y con el GLC funciona bien, pero en Lagged-Fibonacci me daba periodos absurdamente cortos, de unas pocas decenas, cuando ese generador debería tener un periodo enorme. El problema era que ahí el estado no es un número sino la ventana de los últimos k valores, y que se repita un X_i suelto no dice nada.

Por eso reescribí los cuatro generadores bajo una interfaz común: cada uno queda descrito como una terna (estado inicial, función de paso, constante de normalización), donde la función de paso recibe el estado y devuelve el nuevo estado junto con la salida X_i . Así el ciclo se busca sobre el estado completo, que en Lagged-Fibonacci es la ventana de los últimos k valores y en Shift-Register la de los últimos $\max(p, q)$ bits.



```
1 def make_glc(X0, a, c, m):
2     def paso(X):
3         Xn = (a * X + c) % m
4         return Xn, Xn
5     return X0, paso, m
6
7 def make_lagged_fibonacci(semillas, j, k, m):
8     # estado = los ultimos k valores generados
9     def paso(estado):
10        v = (estado[-j] + estado[-k]) % m
11        return estado[1:] + (v,), v
12    return tuple(semillas), paso, m
13
14 def tamano_ciclo(gen, max_iter=MAX_ITER):
15     """(cola, ciclo). (None, None) si no se detecta dentro de max_iter."""
16     estado, paso, _ = gen
17     vistos = {estado: 0}
18     for i in range(1, max_iter + 1):
19         estado, _ = paso(estado)
20         if estado in vistos:
21             return vistos[estado], i - vistos[estado]
22         vistos[estado] = i
23     return None, None
```

Bloque de código 1: Interfaz comun de los generadores y deteccion del tamano de ciclo.

3.3. Pruebas de uniformidad y aleatoriedad (5.1.4)

Implementé el contraste χ^2 con $k = 10$ clases, ecuación (4), y el test de rachas de la ecuación (5), siguiendo los notebooks de clase [6]. Las dos pruebas se aplicaron con $\alpha = 0,05$ a las doce secuencias. Agregué además una decimotercera secuencia de control: el mismo Lagged-Fibonacci, pero inicializado con las 17 primeras salidas del GLC de $m = 2^{48}$ en vez de las semillas originales, para poder separar el efecto del método del efecto de la inicialización.

```
1 def test_chi2(U, k=10, alpha=ALPHA):
2     n = len(U)
3     O, _ = np.histogram(U, bins=k, range=(0, 1))
4     E = n / k
5     T = np.sum((O - E) ** 2 / E)
6     T_critico = stats.chi2.ppf(1 - alpha, k - 1)
7     p_valor = 1 - stats.chi2.cdf(T, k - 1)
8     return T, T_critico, p_valor, T <= T_critico
9
10 def test_rachas(U, alpha=ALPHA):
11     n = len(U)
12     # A[i] = 0 si u_{i+1} < u_i (descendente), 1 en caso contrario
13     A = np.array([0 if U[i - 1] > U[i] else 1 for i in range(1, n)])
14     R = 1 + int(np.sum(A[1:] != A[:-1])) # numero de rachas
15     mu = (2 * n - 1) / 3
16     sigma = np.sqrt((16 * n - 29) / 90)
17     Z = (R - mu) / sigma
18     Z_critico = stats.norm.ppf(1 - alpha / 2)
19     return R, Z, Z_critico, abs(Z) <= Z_critico
```

Bloque de código 2: Pruebas de uniformidad (chi-cuadrado) y de aleatoriedad (rachas).



3.4. Aproximación de integrales por Monte Carlo (5.2)

Desarrollé los ejercicios 3 a 9 de la sección 3.2 del Capítulo 3 de [2]. Cada integral se lleva al dominio $(0, 1)^d$ con los cambios de variable (7) y (8) y se estima con $n = 10.000$ réplicas del estimador (6). Los u_i salen del generador congruencial mixto con $a = 25214903917$, $c = 11$ y $m = 2^{48}$, que fue el que mejor se comportó en 5.1.4; cada ejercicio usa una semilla distinta para trabajar con flujos independientes. Junto a cada estimación reporto el error estándar $s/\sqrt{n}$ y el intervalo de confianza al 95 %, y comparo con el valor exacto (analítico cuando hay forma cerrada, o por cuadratura numérica con `scipy.integrate` cuando no).

Los más laboriosos de plantear fueron el 6, el 7 y el 9, porque hay que transformar un dominio infinito antes de poder muestrear nada; el 9 necesita además el indicador $I\{y < x\}$ que sugiere el propio [2] para llevar la región triangular a $(0, \infty)^2$.

3.5. Generación de variables aleatorias discretas (5.3)

Los cuatro apartados usan el generador congruencial mixto que exige la guía: $X_0 = 2391$, $a = 25214903917$, $c = 11$, $m = 2^{48}$. Los apartados (a), (b) y (c) los resolví con transformada inversa, ecuación (9); en (b) apliqué además la variante eficiente que recorre los valores en orden decreciente de p_j . El (d) va con el método de composición, descomponiendo la masa objetivo según la ecuación (12).

3.6. Simulación *ad hoc* con variables aleatorias generadas (5.4)

Reproduje las secciones 5.1 y 5.2 del Laboratorio 1 (sistema de cola con un servidor y 20 clientes, [7]) cambiando los dados por los generadores propios: los tiempos entre llegadas salen del generador de Poisson($\lambda = 10$) y los de servicio del de Binomial($n = 10$, $p = 0,40$). Igual que en el Laboratorio 1, cada variable aleatoria consume un flujo independiente de números pseudoaleatorios, es decir dos semillas distintas del mismo GLC. Calculé las cinco medidas de desempeño de [7] y, como la conclusión principal del Laboratorio 1 fue que una sola corrida de 20 clientes no alcanza, repetí la simulación diez veces con semillas distintas.

```
1 def genpoisson(u, lam):
2     i, p = 0, np.exp(-lam)
3     F = p
4     while u >= F:
5         p = lam * p / (i + 1)
6         F += p
7         i += 1
8     return i
9
10 def genbinomial(u, n, p):
11     c = p / (1 - p)
12     i, pr = 0, (1 - p) ** n
13     F = pr
14     while u >= F:
15         pr = c * (n - i) / (i + 1) * pr
16         F += pr
17         i += 1
18     return i
```

Bloque de código 3: Generadores de variables aleatorias Poisson y Binomial por transformada inversa.



4. Resultados y análisis

4.1. MidSquare (5.1.1)

La Tabla 1 muestra los primeros y últimos valores u_i de las cinco secuencias generadas con cuadrados medios. El problema del método salta a la vista: con las semillas a (1931) y d (12122) la secuencia colapsa a $u_i = 0$ y ya no sale de ahí, y con la semilla c (4567) se queda dando vueltas sobre el patrón $0,2100 \rightarrow 0,4100 \rightarrow 0,8100 \rightarrow 0,6100$.

i	a (1931)	b (675248)	c (4567)	d (12122)	e (7179345)
1	0.7287	0.9599	0.8574	0.4694	0.4299
2	0.1003	0.3331	0.5134	0.0355	0.8537
3	0.0060	0.9816	0.3579	0.1261	0.8631
4	0.0036	0.5248	0.8092	0.5899	0.9566
5	0.0012	0.4329	0.4804	0.7935	0.1371
			$\vdots$		
96	0.0000	0.6645	0.8100	0.0000	0.6482
97	0.0000	0.6227	0.6100	0.0000	0.1441
98	0.0000	0.7740	0.2100	0.0000	0.7514
99	0.0000	0.0311	0.4100	0.0000	0.5749
100	0.0000	0.9677	0.8100	0.0000	0.5214

Tabla 1: Extracto de las cinco secuencias de $n = 100$ números pseudoaleatorios generadas con MidSquare. Las secuencias de las semillas a y d degeneran al valor absorbente 0; la de la semilla c queda atrapada en un ciclo de cuatro valores.

4.2. Generador congruencial mixto (5.1.2)

La Tabla 2 presenta el extracto del GLC. Aquí el problema ya no es la degeneración sino el periodo corto de los generadores con módulo pequeño. En la columna (a) el valor de la iteración 97 es el mismo de la iteración 1 (0,1016), lo que deja ver un periodo de 32 sin necesidad de calcular nada; en la columna (b) pasa igual, con periodo 48. Las columnas (c) y (d), con módulos $2^{31} - 1$ y 2^{48} , no repiten ningún valor.



i	$\mathbf{a} \ (m = 128)$	$\mathbf{b} \ (m = 48)$	$\mathbf{c} \ (m = 2^{31} - 1)$	$\mathbf{d} \ (m = 2^{48})$	$\mathbf{e} \ (m = 200)$
1	0.1016	0.5000	0.0001	0.8316	0.110
2	0.6797	0.6042	0.7100	0.5264	0.585
3	0.1641	0.2083	0.8229	0.5038	0.960
4	0.8672	0.3125	0.9625	0.0123	0.835
5	0.7266	0.9167	0.9260	0.9113	0.210
			$\vdots$		
96	0.9922	0.8958	0.9978	0.5622	0.335
97	0.1016	0.5000	0.3571	0.1599	0.760
98	0.6797	0.6042	0.9396	0.9668	0.835
99	0.1641	0.2083	0.5011	0.8683	0.210
100	0.8672	0.3125	0.4001	0.1523	0.085

Tabla 2: Extracto de las cinco secuencias generadas con el generador congruencial mixto. En las columnas (a) y (b) la repetición de los valores iniciales de los periodos de 32 y 48.

4.3. Otros generadores: Lagged-Fibonacci y Shift-Register (5.1.3)

De los cuatro métodos propuestos en [3] implementé el Lagged-Fibonacci (con $j = 5$, $k = 17$, $m = 2^{31}$, operación de suma y 17 semillas) y el Shift-Register (con $p = 7$, $q = 3$ y bloques de $n_b = 10$ bits). El resultado del Lagged-Fibonacci me desconcertó: sus primeros valores son prácticamente cero (0,0000, 0,0003, 0,0000, 0,0000, 0,0033, ...) y el mayor de los 100 valores generados apenas llega a 0,5858. Al ver el histograma amontonado contra el eje izquierdo di por hecho que tenía un error en la recurrencia y estuve un buen rato revisando los índices.

No era eso. Las 17 semillas que elegí son números de a lo sumo siete cifras, insignificantes frente a $m = 2^{31} \approx 2,1 \times 10^9$, así que las primeras sumas siguen siendo pequeñas y los $u_i = X_i/m$ tardan cientos de iteraciones en poblar el intervalo $(0, 1)$. El punto siguiente cuantifica el efecto y lo corrige.

4.4. Tamaño de ciclo, uniformidad y aleatoriedad (5.1.4)

La Tabla 3 resume las tres mediciones para las trece secuencias y la Figura 1 muestra las distribuciones empíricas, donde se ve a simple vista lo que el estadístico χ^2 cuantifica.



Generador	Ciclo	Cola	χ^2 calc.	p -valor	¿Uniforme?	R	Z calc.	¿Aleatorio?
MS-a (1931)	1	7	860.60	0.0000	No	2	-15,398	No
MS-b (675248)	1	210	12.40	0.1917	Sí	70	0.878	Sí
MS-c (4567)	4	19	109.40	0.0000	No	48	-4,388	No
MS-d (12122)	1	88	52.20	0.0000	No	54	-2,952	No
MS-e (7179345)	20	848	15.00	0.0909	Sí	66	-0,080	Sí
GLC-a ($m = 128$)	32	0	6.80	0.6579	Sí	69	0.638	Sí
GLC-b ($m = 48$)	48	0	1.20	0.9988	Sí	71	1.117	Sí
GLC-c ($m = 2^{31} - 1$)	$> 4 \times 10^5$	—	12.00	0.2133	Sí	64	-0,558	Sí
GLC-d ($m = 2^{48}$)	$> 4 \times 10^5$	—	4.80	0.8514	Sí	63	-0,798	Sí
GLC-e ($m = 200$)	8	2	22.80	0.0067	No	74	1.835	Sí
Lagged-Fibonacci	$> 4 \times 10^5$	—	628.60	0.0000	No	80	3.271	No
Shift-Register	127	0	2.00	0.9915	Sí	62	-1,037	Sí
Lagged-Fib. (semillas GLC)	$> 4 \times 10^5$	—	5.20	0.8165	Sí	66	-0,080	Sí

Tabla 3: Tamaño de ciclo, prueba de uniformidad (χ^2 con $k = 10$ clases, valor crítico $\chi_{9;0,05}^2 = 16,92$) y prueba de aleatoriedad (rachas, valor crítico $Z_{0,025} = 1,96$) para las trece secuencias de $n = 100$ números. La última fila es una secuencia de control: el mismo Lagged-Fibonacci inicializado con salidas del GLC-d.



Distribución empírica de las secuencias de 100 números pseudoaleatorios
(línea roja = frecuencia esperada bajo uniformidad)

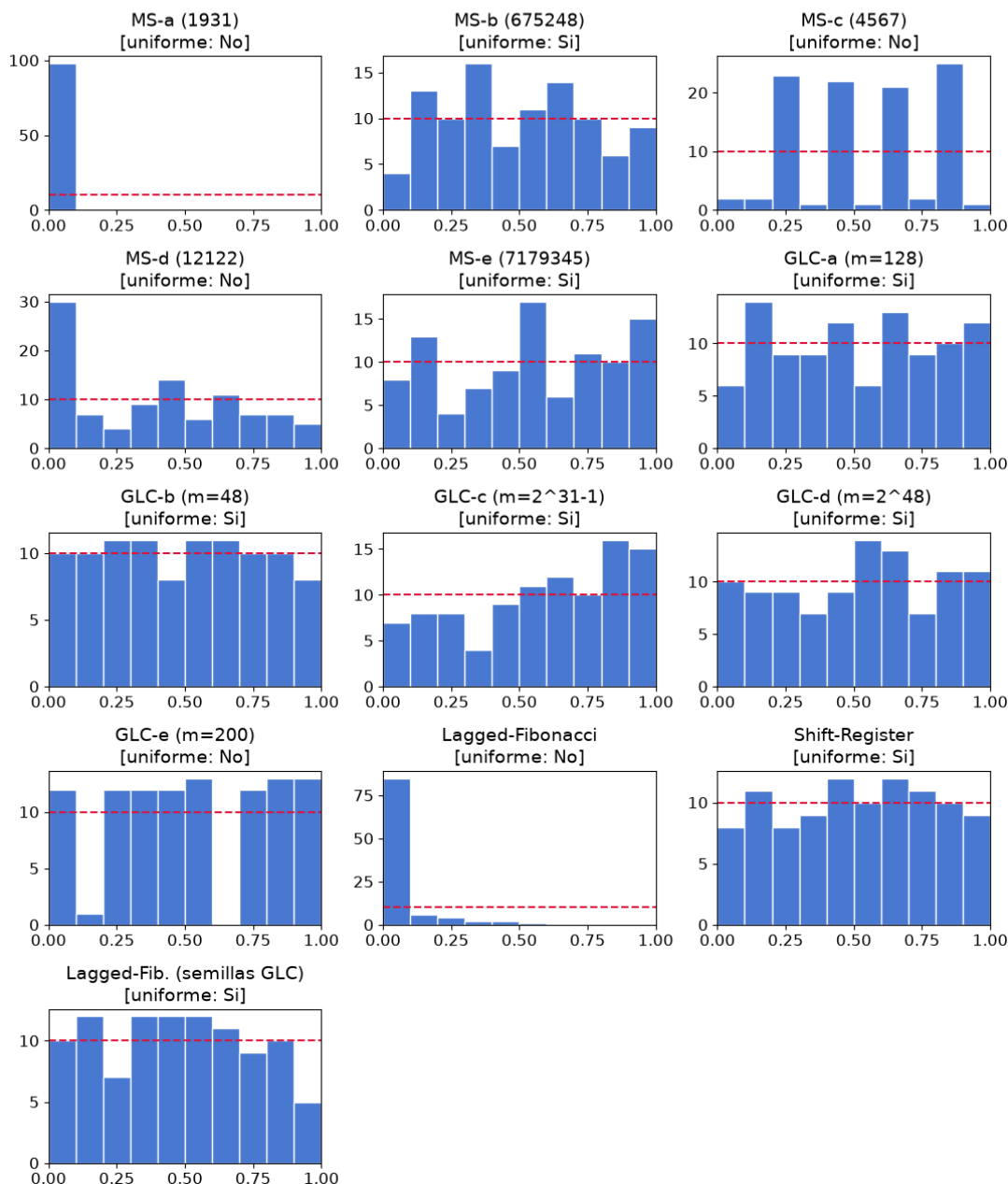


Figura 1: Distribución empírica de cada secuencia de 100 números pseudoaleatorios sobre $k = 10$ clases. La línea roja marca la frecuencia esperada bajo uniformidad ($E_i = 10$).

¿Cuáles son los peores generadores de números aleatorios y por qué?

Los peores son MidSquare con las semillas a (1931), c (4567) y d (12122), y el GLC-e ($X_0 = 3$, $a = 5$, $c = 7$, $m = 200$). Detrás va el Lagged-Fibonacci con la inicialización original, aunque por un motivo distinto.

MidSquare con las semillas a y d tiene ciclo de longitud 1: cae en el estado absorbente $X = 0$ en los pasos 7 y 88 respectivamente y desde ahí repite siempre el mismo valor. Falla las



dos pruebas de forma aplastante ($\chi^2 = 860,60$ y $52,20$ frente al crítico $16,92$; $Z = -15,398$ y $-2,952$). Con la semilla c entra en un ciclo de cuatro valores, así que las 100 observaciones se reducen a cuatro números distintos y el histograma queda en cuatro barras aisladas.

El GLC-e tiene un periodo de 8 valores porque no cumple el teorema de Hull-Dobell: $m = 200 = 2^3 \cdot 5^2$ y $a - 1 = 4$ no es divisible por 5. Ni siquiera alcanza el periodo máximo de su propio módulo, que ya era pequeño. Por eso falla uniformidad ($\chi^2 = 22,80 > 16,92$), aunque pase la de rachas por poco.

El Lagged-Fibonacci original es un caso aparte: tiene un periodo enorme y aun así falla las dos pruebas ($\chi^2 = 628,60$, $Z = 3,271$), por el problema de inicialización que ya describí. La fila de control lo confirma: con los mismos j , k y m , pero sembrado con salidas del GLC-d, el mismo generador las pasa ($\chi^2 = 5,20$, $Z = -0,080$).

Salvo ese último caso, todos comparten el mismo defecto: periodo corto o estados absorbentes. Un generador cuyo periodo es menor que la cantidad de números que la simulación va a consumir termina reutilizando la misma secuencia y deja de servir como muestra i.i.d. $U(0, 1)$. Las réplicas dejan de ser independientes y los intervalos de confianza que se calculen con ellas ya no significan nada.

¿Cuáles son los mejores generadores de números aleatorios y por qué?

Los mejores son el GLC-c ($X_0 = 13$, $a = 16807$, $c = 0$, $m = 2^{31} - 1$) y el GLC-d ($X_0 = 29022024$, $a = 25214903917$, $c = 11$, $m = 2^{48}$): son los únicos que juntan periodo máximo, uniformidad y aleatoriedad. El GLC-c es el conocido *Minimal Standard*; su módulo es el primo de Mersenne $2^{31} - 1$ y $a = 16807 = 7^5$ es raíz primitiva módulo m , de modo que su periodo es $m - 1 = 2^{31} - 2 \approx 2,15 \times 10^9$. El GLC-d es el de `java.util.Random` y cumple las tres condiciones de Hull-Dobell: $\gcd(11, 2^{48}) = 1$; $a - 1 = 25214903916$ es par y 2 es el único factor primo de m ; y $a - 1$ es divisible por 4 igual que m . Su periodo es entonces $m = 2^{48} \approx 2,81 \times 10^{14}$. De ahí que en los dos casos el algoritmo de detección llegara al tope de 4×10^5 iteraciones sin encontrar repetición. Las dos pruebas las pasan con holgura: $\chi^2 = 12,00$ y $4,80$ frente a $16,92$, y $|Z| = 0,558$ y $0,798$ frente a $1,96$.

El Shift-Register y el GLC-b también pasan las dos pruebas, con estadísticos incluso mejores ($\chi^2 = 2,00$ y $1,20$), y aun así no sirven: sus periodos son de 127 y 48 valores. El GLC-b es el caso más curioso, porque tiene periodo máximo (cumple Hull-Dobell con $m = 48$) y por construcción recorre una sola vez cada uno de sus 48 estados, lo que produce un histograma demasiado parejo para venir del azar. O sea que pasar las pruebas sobre una muestra corta no basta: el periodo tiene que ser varios órdenes de magnitud mayor que la cantidad de valores que la simulación va a consumir. Por eso escogí el GLC-d para el resto del laboratorio.

4.5. Uso de números aleatorios para evaluar integrales (5.2)

La Tabla 4 reúne los resultados de los siete ejercicios con $n = 10.000$ réplicas y la Figura 2 muestra cómo evoluciona el estimador conforme crece n .



Ej.	Integral	Estimación MC	Error est.	IC 95 %	Valor exacto	Error rel.
3	$\int_0^1 e^{e^x} dx$	6.3182	0.0328	(6.2540, 6.3825)	6.3166	0.03 %
4	$\int_0^1 (1-x^2)^{3/2} dx$	0.5858	0.0033	(0.5793, 0.5923)	0.5890 ($3\pi/16$)	0.55 %
5	$\int_{-2}^2 e^{x+x^2} dx$	93.9143	2.4627	(89.0874, 98.7413)	93.1628	0.81 %
6	$\int_0^\infty x(1+x^2)^{-2} dx$	0.5038	0.0034	(0.4972, 0.5104)	0.5000 ($1/2$)	0.76 %
7	$\int_{-\infty}^\infty e^{-x^2} dx$	1.7839	0.0141	(1.7562, 1.8116)	1.7725 ($\sqrt{\pi}$)	0.65 %
8	$\int_0^1 \int_0^1 e^{(x+y)^2} dy dx$	4.9210	0.0599	(4.8036, 5.0384)	4.8992	0.45 %
9	$\int_0^\infty \int_0^x e^{-(x+y)} dy dx$	0.5040	0.0073	(0.4897, 0.5183)	0.5000 ($1/2$)	0.81 %

Tabla 4: Aproximación por Monte Carlo de los ejercicios 3 a 9 del Capítulo 3 de [2] con $n = 10.000$ réplicas generadas por el GLC de $m = 2^{48}$. El valor exacto es analítico cuando existe forma cerrada y por cuadratura numérica en los demás casos.

Convergencia del estimador Monte Carlo hacia el valor exacto de la integral

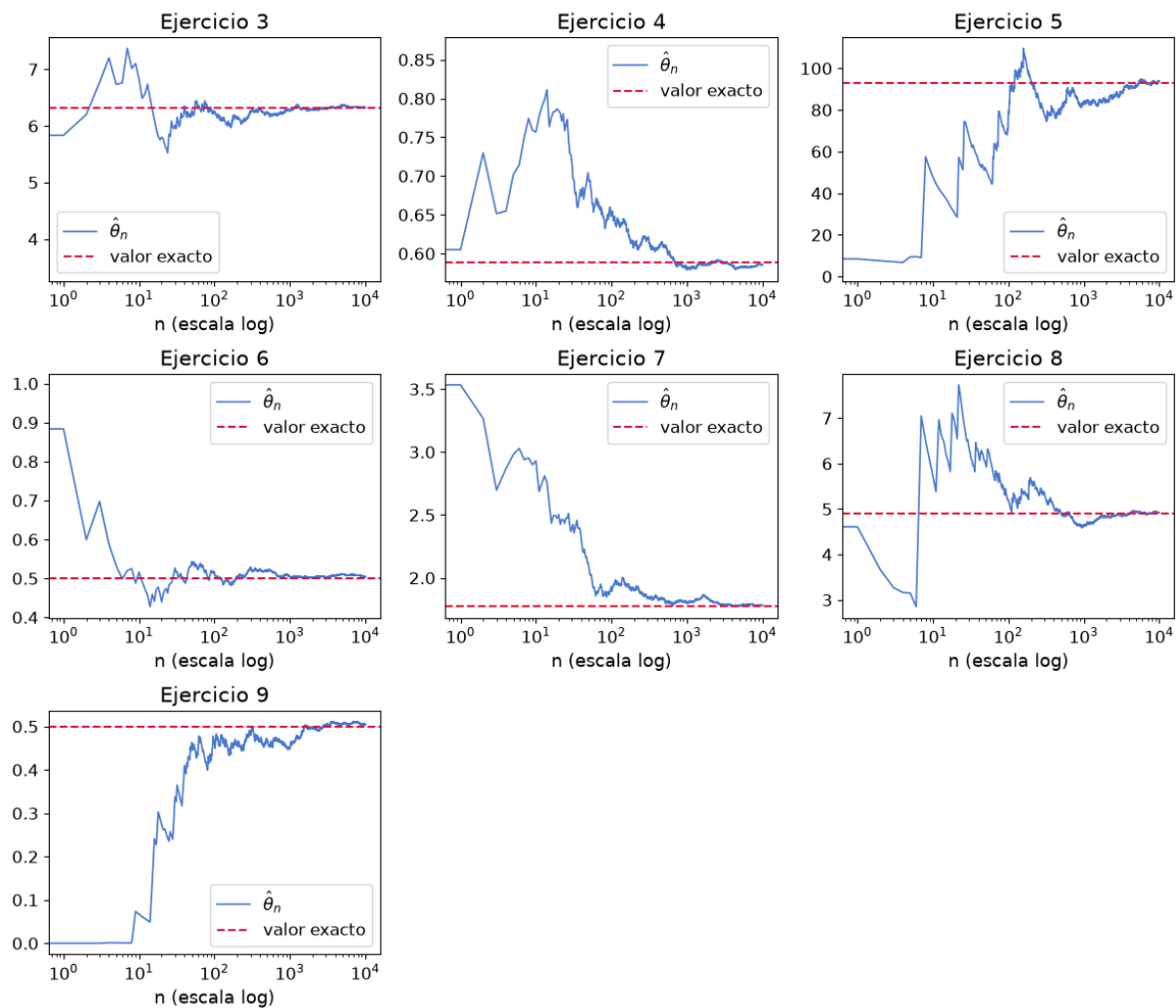


Figura 2: Convergencia del estimador $\hat{\theta}_n$ hacia el valor exacto (línea roja) conforme aumenta el número de réplicas. Eje horizontal en escala logarítmica.



Las siete estimaciones quedaron a menos del 1 % del valor exacto y en todos los casos el intervalo de confianza al 95 % contiene el valor verdadero, lo que valida los cambios de variable y también el generador. En los ejercicios 6 y 9, donde el valor exacto es $1/2$, el método lo recuperó con errores de 0,0038 y 0,0040.

Las gráficas de convergencia muestran la característica más importante del método, que es al mismo tiempo su mayor limitación: el error decrece como $\sigma_g/\sqrt{n}$, así que para ganar un dígito decimal de precisión hay que multiplicar el número de réplicas por 100. También se nota que la velocidad de convergencia depende mucho de la varianza del integrando transformado. El ejercicio 5 es el más inestable, con oscilaciones visibles incluso pasadas las mil réplicas y el mayor error estándar de la tabla (2,4627): la función e^{x+x^2} crece hasta $e^6 \approx 403$ en $x = 2$, de modo que unas pocas réplicas cercanas al extremo derecho dominan la suma. Los ejercicios 6, 7 y 9 tienen soporte infinito y el cambio de variable $y = 1/(x+1)$ comprime toda la cola cerca de $y \rightarrow 0$, donde el factor $1/y^2$ amplifica la varianza; por eso el ejercicio 9 arranca en cero — ninguna de las primeras réplicas cumple $y < x$ — y necesita alrededor de mil réplicas para estabilizarse. Los ejercicios 4 y 6, con integrandos acotados y suaves, convergen rápido.

4.6. Generación de variables aleatorias discretas (5.3)

(a) $p_1 = 0,20$, $p_2 = 0,15$, $p_3 = 0,25$, $p_4 = 0,40$

Generé los primeros 100 valores por transformada inversa. La Tabla 5 (izquierda) compara las frecuencias observadas con las esperadas.

(b) $P\{X = 1\} = 0,3$, $P\{X = 2\} = 0,2$, $P\{X = 3\} = 0,35$, $P\{X = 4\} = 0,15$

Este apartado corresponde al ejercicio 3 del Capítulo 4 de [2], que pide un *algoritmo eficiente*. Siguiendo la observación 2 de la sección 4.1, basta con recorrer los valores en orden decreciente de p_j , es decir $3(0,35) \rightarrow 1(0,30) \rightarrow 2(0,20) \rightarrow 4(0,15)$, porque el tiempo de ejecución es proporcional al número de intervalos examinados. El número medio de comparaciones baja de 2,350 a 2,150, una reducción del 8,5 %. El algoritmo queda así:

Generar U . Si $U < 0,35$ hacer $X = 3$ y terminar. Si $U < 0,65$ hacer $X = 1$ y terminar.
Si $U < 0,85$ hacer $X = 2$ y terminar. En caso contrario, hacer $X = 4$.

(a) $n = 100$				(b) $n = 100$			
$X = j$	p_j	Esp.	Obs.	$X = j$	p_j	Esp.	Obs.
1	0.20	20	21	1	0.30	30	30
2	0.15	15	14	2	0.20	20	18
3	0.25	25	27	3	0.35	35	35
4	0.40	40	38	4	0.15	15	17

Tabla 5: Frecuencias esperadas y observadas de los 100 valores generados por transformada inversa en los apartados (a) y (b) de 5.3.

En los dos apartados las frecuencias observadas se desvían de las esperadas a lo sumo en dos unidades. Es una diferencia normal para $n = 100$: la desviación estándar de una frecuencia binomial con $p = 0,25$ es $\sqrt{100 \cdot 0,25 \cdot 0,75} \approx 4,33$.



(c) $p_1 = 1/3$, $p_2 = 2/3$ con $n = 100$, 1000 y 10000

n	Número de unos	Proporción de unos	Error absoluto	Error estándar teórico
100	34	0.3400	0.0067	0.0471
1000	328	0.3280	0.0053	0.0149
10000	3316	0.3316	0.0017	0.0047

Tabla 6: Proporción de valores iguales a 1 para la masa $p_1 = 1/3$, $p_2 = 2/3$, comparada con el valor teórico $p_1 = 0,3333$ y con el error estándar teórico $\sqrt{p_1(1-p_1)/n}$.

La proporción de unos converge a $1/3$ conforme aumenta n , como predice la ley fuerte de los grandes números: el error absoluto pasa de 0,0067 a 0,0017 al multiplicar n por 100. En los tres casos el error observado queda por debajo del error estándar teórico $\sqrt{p_1(1-p_1)/n}$, señal de que el generador no introduce sesgo. Vale aclarar que el error no baja de forma monótona con n : para $n = 1000$ es apenas menor que para $n = 100$. Eso es normal, porque el error es una variable aleatoria y lo que disminuye con seguridad es su desviación estándar, no cada realización particular.

(d) Método de composición

Para la masa $p = (0,06, 0,06, 0,06, 0,06, 0,06, 0,15, 0,13, 0,14, 0,15, 0,13)$ sobre $j = 1, \dots, 10$ se buscó una descomposición de la forma (12) tomando como q la uniforme discreta en $1, \dots, 10$, es decir $q_j = 1/10$. Para $j \leq 5$ debe cumplirse $0,06 = \alpha/10$, de donde

$$\alpha = 0,6 \quad \text{y} \quad r_j = \frac{p_j - 0,06}{0,4} \quad \text{para } j \geq 6$$

lo que da $r_6 = 0,225$, $r_7 = 0,175$, $r_8 = 0,200$, $r_9 = 0,225$ y $r_{10} = 0,175$, que efectivamente suman 1. El algoritmo obtenido es:

PASO 1: Generar dos números aleatorios U_1 y U_2 .

PASO 2: Si $U_1 < 0,6$, hacer $X = \text{Ent}(10 U_2) + 1$ y terminar.

PASO 3: En caso contrario, obtener $X \in \{6, \dots, 10\}$ por transformada inversa sobre r_j usando U_2 .

La ventaja de esta descomposición es que en el 60 % de los casos X se resuelve con una multiplicación y un truncamiento, sin recorrer ningún intervalo, y en el 40 % restante hay que revisar cinco intervalos en vez de diez. Generé 10.000 valores para contrastar la distribución empírica con la teórica (Tabla 7).



$X = j$	p_j teórica	Frec. esperada	Frec. observada	Frec. relativa
1	0.06	600	577	0.0577
2	0.06	600	627	0.0627
3	0.06	600	586	0.0586
4	0.06	600	561	0.0561
5	0.06	600	623	0.0623
6	0.15	1500	1526	0.1526
7	0.13	1300	1274	0.1274
8	0.14	1400	1402	0.1402
9	0.15	1500	1557	0.1557
10	0.13	1300	1267	0.1267

Tabla 7: Distribución teórica y empírica de 10.000 valores generados con el método de composición.

La prueba χ^2 de bondad de ajuste sobre esos 10.000 valores da $\chi^2 = 9,817$ con un p -valor de 0,3655, muy por encima de $\alpha = 0,05$, así que no se rechaza que los valores generados vengan de la distribución objetivo. La Figura 3 compara las distribuciones teórica y empírica de los tres apartados.

Distribución de probabilidad teórica vs empírica

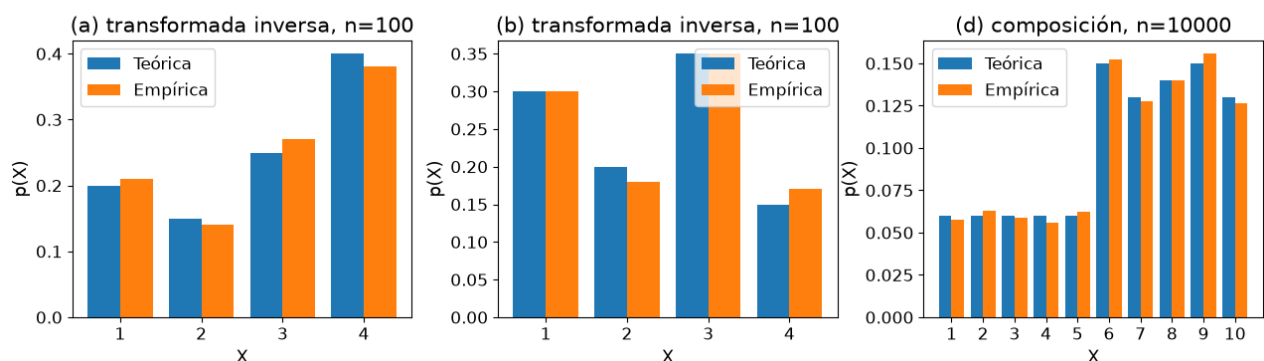


Figura 3: Distribución de probabilidad teórica frente a la empírica en los apartados (a), (b) y (d) de 5.3. El ajuste mejora notoriamente al aumentar el número de valores generados.

Comparando los tres paneles se nota el efecto del tamaño de muestra: con $n = 100$ las barras teórica y empírica difieren de forma visible, hasta 0,02 en el apartado (a), mientras que con $n = 10.000$ en el apartado (d) las diferencias son casi imperceptibles. Es lo mismo que ya habíamos visto en el Laboratorio 1.

4.7. Simulación *ad hoc* con generación de variables aleatorias discretas (5.4)

La Tabla 8 recoge la simulación de 20 clientes con los tiempos entre llegadas generados por el algoritmo de Poisson($\lambda = 10$) y los de servicio por el de Binomial($n = 10$, $p = 0,40$).



Customer	Time Between Arrivals	Arrival Time	Service Time	Service Begins	Time Service Ends	Time in System	Idle Time	Time in Queue
1	—	0	4	0	4	4	0	0
2	12	12	7	12	19	7	8	0
3	11	23	5	23	28	5	4	0
4	12	35	3	35	38	3	7	0
5	12	47	2	47	49	2	9	0
6	9	56	3	56	59	3	7	0
7	11	67	6	67	73	6	8	0
8	14	81	3	81	84	3	8	0
9	9	90	3	90	93	3	6	0
10	12	102	5	102	107	5	9	0
11	8	110	4	110	114	4	3	0
12	4	114	3	114	117	3	0	0
13	12	126	3	126	129	3	9	0
14	12	138	5	138	143	5	9	0
15	6	144	6	144	150	6	1	0
16	7	151	6	151	157	6	1	0
17	7	158	4	158	162	4	1	0
18	6	164	3	164	167	3	2	0
19	7	171	3	171	174	3	4	0
20	9	180	4	180	184	4	6	0
						82	102	0

Tabla 8: Simulación del sistema de fila bancaria para 20 clientes con tiempos entre llegadas Poisson($\lambda = 10$) y tiempos de servicio Binomial($n = 10, p = 0,40$), ambos generados por transformada inversa a partir del GLC de $m = 2^{48}$.

Las medidas de desempeño calculadas a partir de la Tabla 8 son:

Average time in system = $82/20 = 4,10$ minutos.

Percent idle time = $(102/184)(100) = 55,43$ %.

Average waiting time per customer = $0/20 = 0,00$ minutos.

Fraction having to wait = $0/20 = 0,00$.

Average waiting time of those who waited = no aplica (ningún cliente esperó).

Como en el Laboratorio 1 quedó claro que una sola corrida de 20 clientes no alcanza para caracterizar el sistema, repetí la simulación diez veces con semillas distintas (Tabla 9) para ver si lo observado era estructural o casualidad de una corrida.



Run	Average time in system	Percent idle time	Average waiting time per customer	Fraction having to wait	Average waiting time of those who waited
1	4.10	55.43 %	0.00	0.00	0.0
2	4.25	55.73 %	0.00	0.00	0.0
3	3.50	66.18 %	0.00	0.00	0.0
4	4.15	56.61 %	0.05	0.05	1.0
5	4.90	51.52 %	0.10	0.10	1.0
6	4.05	59.90 %	0.00	0.00	0.0
7	4.15	58.50 %	0.00	0.00	0.0
8	3.70	62.18 %	0.05	0.05	1.0
9	3.15	69.27 %	0.00	0.00	0.0
10	4.15	56.84 %	0.05	0.05	1.0
Media	4.01	59.22 %	0.025	0.025	0.40

Tabla 9: Medidas de desempeño de 10 corridas de 20 clientes con tiempos entre llegadas Poisson(10) y tiempos de servicio Binomial(10; 0,40).

Variables aleatorias generadas por transformada inversa (5000 réplicas)

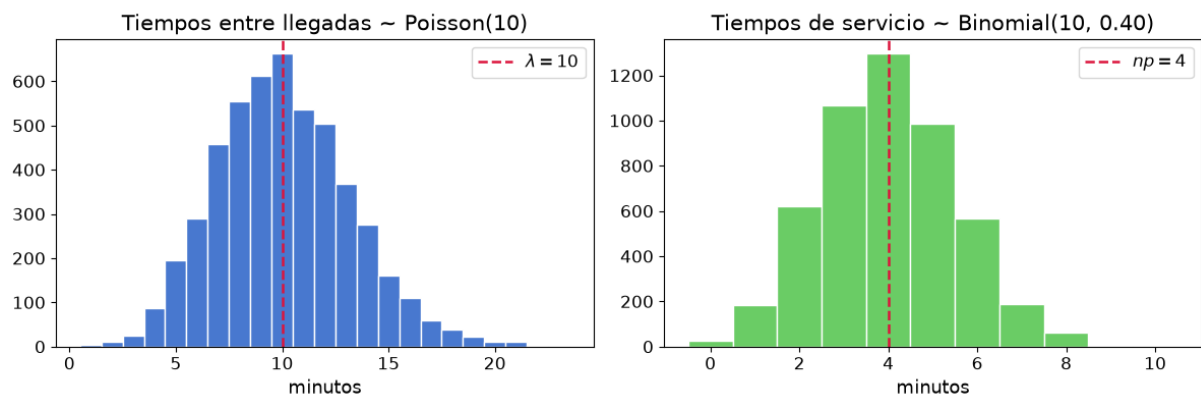


Figura 4: Distribución empírica de 5.000 tiempos entre llegadas Poisson($\lambda = 10$) y 5.000 tiempos de servicio Binomial($n = 10$, $p = 0,40$) generados por transformada inversa. La línea roja marca la media teórica de cada distribución.

¿Qué puede decir de los resultados obtenidos? ¿Qué similitudes o diferencias se presentan en esta simulación en comparación con el Laboratorio 1?

La mecánica del sistema es la misma del Laboratorio 1 y el tiempo promedio en el sistema salió casi idéntico: 4,10 minutos en la corrida principal frente a los 4,35 de la primera corrida del Laboratorio 1, y 4,01 como media de las diez corridas frente a 4,48. También se repite lo de la variabilidad: entre corridas de 20 clientes el tiempo promedio en el sistema osciló entre 3,15 y 4,90 minutos, y por eso hubo que repetir la simulación para poder afirmar algo.

La diferencia grande es que el sistema quedó mucho más descongestionado, y la causa está



en las medias de las distribuciones, no en la simulación. En el Laboratorio 1 los tiempos entre llegadas eran uniformes discretos en $\{1, \dots, 10\}$ con media 5,5 y los de servicio uniformes discretos en $\{1, \dots, 6\}$ con media 3,5, lo que da un factor de utilización $\rho = 3,5/5,5 = 0,64$. Aquí las llegadas son Poisson(10) con media 10 y los servicios Binomial(10; 0,40) con media $np = 4$, o sea $\rho = 4/10 = 0,40$. Con un ρ tan bajo casi nunca llega un cliente mientras el anterior sigue en atención. Por eso en la corrida principal ningún cliente esperó y el servidor estuvo inactivo el 55,43 % del tiempo. La Tabla 10 resume el contraste.

Medida (media de 10 corridas)	Laboratorio 1	Laboratorio 2
Distribución de los tiempos entre llegadas	$\mathcal{U}\{1, \dots, 10\}$	Poisson(10)
Distribución de los tiempos de servicio	$\mathcal{U}\{1, \dots, 6\}$	Binomial(10; 0,40)
Factor de utilización ρ	0.64	0.40
Average time in system	4.48 min	4.01 min
Percent idle time	34.80 %	59.22 %
Average waiting time per customer	1.13 min	0.03 min
Fraction having to wait	0.37	0.03
Average waiting time of those who waited	3.46 min	0.40 min

Tabla 10: Comparación de las medidas de desempeño promedio de las 10 corridas de 20 clientes entre el Laboratorio 1 (datos, distribuciones uniformes discretas) y el Laboratorio 2 (generadores propios de Poisson y Binomial).

Hay un segundo efecto, más sutil, que tiene que ver con la forma de las distribuciones. La uniforme discreta del Laboratorio 1 reparte la probabilidad por igual entre todos sus valores, mientras que la Poisson y la Binomial concentran la masa alrededor de la media, como se ve en la Figura 4 ($\sigma = \sqrt{\lambda} \approx 3,16$ y $\sigma = \sqrt{np(1-p)} \approx 1,55$). Los tiempos de servicio resultan entonces más regulares que en el Laboratorio 1, donde la uniforme en $\{1, \dots, 6\}$ tiene $\sigma \approx 1,71$ sobre una media menor, y eso reduce todavía más la formación de cola: la congestión depende tanto de ρ como de la variabilidad de los tiempos. Otro detalle es que las dos distribuciones admiten el valor 0 — dos clientes llegando en el mismo instante, o un servicio instantáneo —, algo que las uniformes en $\{1, \dots, 10\}$ y $\{1, \dots, 6\}$ no podían producir.

En lo operativo, con estos parámetros el servidor está sobredimensionado. Con el mismo criterio del Laboratorio 1, comparar el costo de hacer esperar a los clientes contra el ahorro de un servidor más lento, aquí sí convendría un servidor más lento, o dedicarlo a otras tareas más de la mitad del tiempo.

5. Conclusiones

- Programé desde cero cuatro métodos de generación de números pseudoaleatorios (MidSquare, congruencial mixto, Lagged-Fibonacci y Shift-Register) y produje con ellos trece secuencias de 100 números. MidSquare confirmó en la práctica su defecto histórico: tres de las cinco semillas ensayadas terminaron en ciclos de longitud 1 a 4, y con la semilla 1931 el generador cayó en el estado absorbente $X = 0$ después de siete iteraciones.
- Medir el tamaño de ciclo obliga a trabajar con el estado completo del generador y no con su salida; fue el punto que más me costó resolver del laboratorio. Los periodos medidos van desde 1 (MidSquare degenerado) y 8 (GLC con $m = 200$) hasta valores que no aparecen en 4×10^5 iteraciones para los generadores de módulo grande, cuyos periodos teóricos son



$2^{31} - 2 \approx 2,15 \times 10^9$ y $2^{48} \approx 2,81 \times 10^{14}$ por cumplirse, respectivamente, la condición de raíz primitiva y las tres condiciones de Hull-Dobell.

- Las pruebas de uniformidad (χ^2 , valor crítico 16,92) y de aleatoriedad (rachas, valor crítico 1,96) sirvieron para clasificar los generadores, pero también mostraron hasta dónde llegan: el GLC de $m = 48$ obtuvo el mejor χ^2 de todo el laboratorio (1,20) con un periodo de apenas 48 valores, justamente porque recorre una sola vez cada uno de sus estados. Pasar las pruebas sobre una muestra corta es necesario, pero no suficiente; el periodo debe superar por varios órdenes de magnitud la cantidad de números que la simulación consume. Los mejores del laboratorio fueron el GLC de $m = 2^{31} - 1$ y el de $m = 2^{48}$.
- El fallo del Lagged-Fibonacci ($\chi^2 = 628,60$, $Z = 3,271$) resultó ser un problema de inicialización y no del método: al sembrarlo con las salidas de un buen GLC en lugar de con números pequeños frente a $m = 2^{31}$, el mismo generador pasó las dos pruebas ($\chi^2 = 5,20$, $Z = -0,080$). La calidad de un generador depende de sus parámetros y también de su semilla.
- Monte Carlo aproximó las siete integrales de los ejercicios 3 a 9 del Capítulo 3 de [2] con errores relativos entre 0,03 % y 0,81 % usando $n = 10.000$ réplicas, y en los siete casos el intervalo de confianza al 95 % contuvo el valor exacto. Las curvas de convergencia dejan ver que el error decrece como $\sigma_g/\sqrt{n}$ y que la velocidad depende de la varianza del integrando transformado: el ejercicio 5, con e^{x+x^2} creciendo hasta e^6 , dio un error estándar de 2,4627, dos órdenes de magnitud mayor que el de los integrandos acotados.
- La transformada inversa reprodujo bien las distribuciones objetivo, con desviaciones máximas de dos unidades sobre 100 valores en los apartados (a) y (b); la proporción de unos del apartado (c) convergió de 0,3400 a 0,3316 frente al valor teórico $1/3$ al pasar de $n = 100$ a $n = 10.000$. El método de composición, con $\alpha = 0,6$ y q uniforme discreta en $1, \dots, 10$, resolvió el 60 % de los casos sin recorrer intervalos, y su bondad de ajuste sobre 10.000 valores dio $\chi^2 = 9,817$ con p -valor 0,3655.
- Al repetir la simulación *ad hoc* del Laboratorio 1 con tiempos entre llegadas Poisson ($\lambda = 10$) y tiempos de servicio Binomial ($n = 10$, $p = 0,40$), el tiempo promedio en el sistema se mantuvo casi igual (4,01 frente a 4,48 minutos), pero el porcentaje de tiempo inactivo pasó de 34,80 % a 59,22 % y la fracción de clientes que esperó cayó de 0,37 a 0,03. La explicación no está en el método de generación sino en los parámetros: el factor de utilización bajó de $\rho = 0,64$ a $\rho = 0,40$, y la Poisson y la Binomial concentran la masa alrededor de su media mientras que las uniformes discretas la reparten por igual, lo que reduce todavía más la formación de cola.

Bibliografía

- [1] A. A. Cruz Roa, “Números Aleatorios,” 2025, Diapositivas de clase, Simulación Computacional, Universidad de los Llanos.
- [2] S. M. Ross, *Simulación*, 2nd ed. México D.F., México: Pearson Educación, 1999.
- [3] A. M. Mancilla Herrera, “Números aleatorios. Historia, teoría y aplicaciones,” *Ingeniería y Desarrollo*, no. 8, pp. 49–69, 2000, Universidad del Norte, Barranquilla, Colombia.



- [4] D. Ríos, S. Ríos, and J. Martín, *Simulación. Métodos y aplicaciones*. Bogotá, Colombia: Alfaomega, 2000.
- [5] A. A. Cruz Roa, “Generación de Variables Aleatorias Discretas,” 2025, Diapositivas y Notebook de clase, Simulación Computacional, Universidad de los Llanos.
- [6] —, “Test o contrastes para números aleatorios,” 2025, Diapositivas y Notebooks de clase, Simulación Computacional, Universidad de los Llanos.
- [7] J. Banks, *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice*. New York, NY, USA: John Wiley & Sons, Inc., 1998.